

707. Design Linked List

Solved

Medium

Topics

Companies

Design your implementation of the linked list. You can choose to use a singly or doubly linked list.

A node in a singly linked list should have two attributes: `val` and `next`. `val` is the value of the current node, and `next` is a pointer/reference to the next node. If you want to use the doubly linked list, you will need one more attribute `prev` to indicate the previous node in the linked list. Assume all nodes in the linked list are **0-indexed**.

Implement the `MyLinkedList` class:

- `MyLinkedList()` Initializes the `MyLinkedList` object.
- `int get(int index)` Get the value of the `indexth` node in the linked list. If the index is invalid, return `-1`.
- `void addAtHead(int val)` Add a node of value `val` before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.
- `void addAtTail(int val)` Append a node of value `val` as the last element of the linked list.
- `void addAtIndex(int index, int val)` Add a node of value `val` before the `indexth` node in the linked list. If `index` equals the length of the linked list, the node will be appended to the end of the linked list. If `index` is greater than the length, the node **will not be inserted**.
- `void deleteAtIndex(int index)` Delete the `indexth` node in the linked list, if the index is valid.

Code

Java Auto

```

1  class MyLinkedList {
2
3      class ListNode {
4          int val;
5          ListNode next;
6
7          ListNode(int val) {
8              this.val = val;
9          }
10     }
11
12     ListNode head;
13     int size;
14
15     public MyLinkedList() {
16         head = null;
    
```

Saved

Ln 97, Col 2

☒ Testcase | [> Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1

Input

["MyLinkedList","addAtHead","deleteAtIndex","addAtTail","get"]

[[],[1],[0],[2],[0]]

Output

[null,null,null,null,2]

83. Remove Duplicates from Sorted List

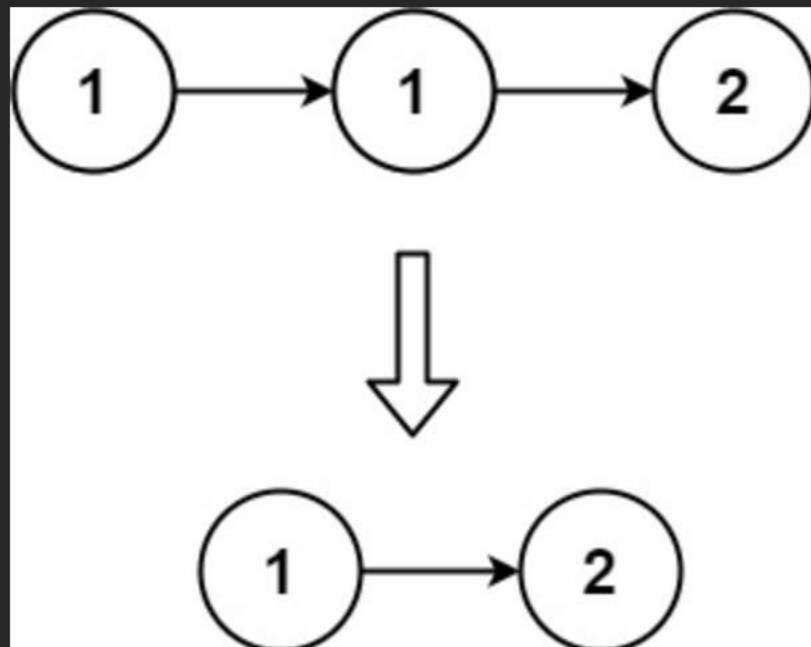
Easy

Topics

Companies

Given the `head` of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list **sorted** as well.

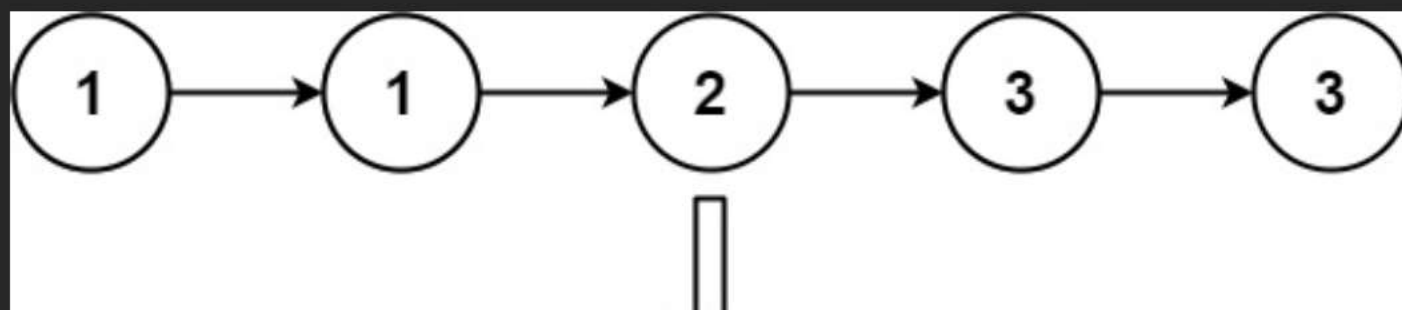
Example 1:



Input: head = [1,1,2]

Output: [1,2]

Example 2:



Code

Java Auto

```

10  */
11  class Solution {
12      public ListNode deleteDuplicates(ListNode head) {
13          ListNode temp = head;
14
15          while (temp != null && temp.next != null) {
16              if (temp.val == temp.next.val) {
17                  temp.next = temp.next.next; // Remove duplicate
18              } else {
19                  temp = temp.next; // Move to next node
20              }
21          }
22
23          return head;
24      }
25  }
  
```

Saved

Ln 25, Col 2

☒ Testcase |
 [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

Input

head =
[1,1,2]

Output

[1,2]

142. Linked List Cycle II

Medium

Topics

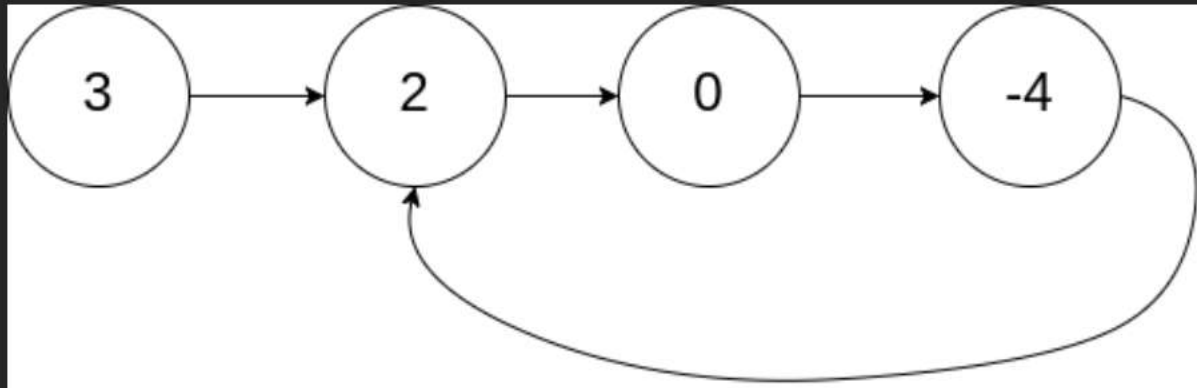
Companies

Given the `head` of a linked list, return *the node where the cycle begins*. If there is no cycle, return `null`.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to (**0-indexed**). It is `-1` if there is no cycle. **Note that `pos` is not passed as a parameter**.

Do not modify the linked list.

Example 1:



Input: `head = [3,2,0,-4]`, `pos = 1`

Output: tail connects to node index 1

Explanation: There is a cycle in the linked list, where tail connects to the second node.

Example 2:



15.3K



278



32 Online

Code

Java | Auto



```
11  */
12  public class Solution {
13      public ListNode detectCycle(ListNode head) {
14          ListNode slow = head;
15          ListNode fast = head;
16          while (fast != null && fast.next != null) {
17              slow = slow.next;
18              fast = fast.next.next;
19
20              if (slow == fast) {
21                  ListNode temp = head;
22
23                  while (temp != slow) {
24                      temp = temp.next;
25                      slow = slow.next;
26                  }
27              }
```

Saved

Ln 15, Col 30

☒ Testcase | ☒ Test Result**Accepted** Runtime: 0 ms☒ Case 1☒ Case 2☒ Case 3

Input

head =

`[3,2,0,-4]`

pos =

`1`

Output

Print Diagonally



Difficulty: **Easy**

Accuracy: **66.11%**

Submissions: **83K+**

Points: **2**

Average Time: **10m**

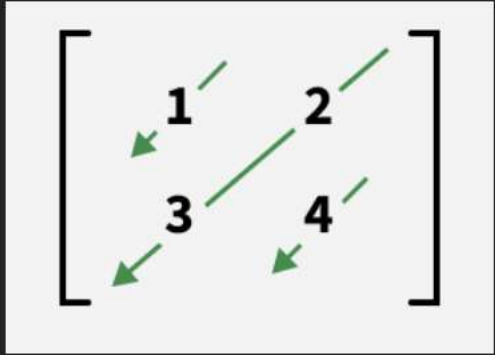
Give a $n \times n$ square matrix `mat[][]`, return all the elements of its anti-diagonals from **top to bottom**.

Examples :

Input: $n = 2$, `mat[][] = [[1, 2], [3, 4]]`

Output: `[1, 2, 3, 4]`

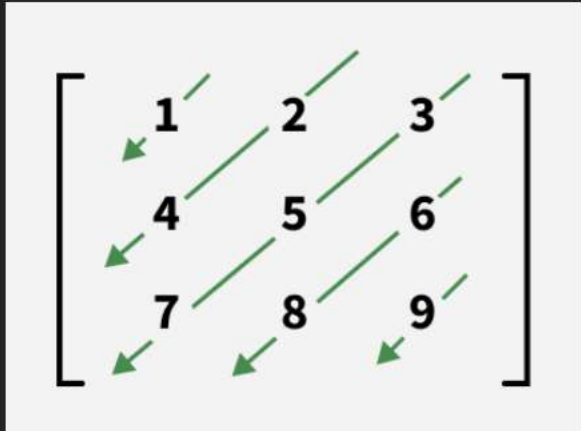
Explanation:



Input: $n = 3$, `mat[][] = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`

Output: `[1, 2, 4, 3, 5, 7, 6, 8, 9]`

Explanation:



```
1 class Solution {
2     public ArrayList<Integer> diagView(int[][] mat) {
3         int n = mat.length;
4         ArrayList<Integer> ans = new ArrayList<>();
5         for (int col = 0; col < n; col++) {
6             int i = 0;
7             int j = col;
8
9             while (i < n && j >= 0) {
10                ans.add(mat[i][j]);
11                i++;
12                j--;
13            }
14        }
15        for (int row = 1; row < n; row++) {
16            int i = row;
17            int j = n - 1;
18
19            while (i < n && j >= 0) {
20                ans.add(mat[i][j]);
21                i++;
22                j--;
23            }
24        }
25
26        return ans;
27    }
28 }
```



[Custom Input](#)

Compile & Run

Submit

867. Transpose Matrix

Easy

Topics

Companies

Hint

Given a 2D integer array `matrix`, return *the transpose of* `matrix`.

The **transpose** of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.

2	4	-1
-10	5	11
18	-7	6

→

2	-10	18
4	5	-7
-1	11	6

Example 1:

Input: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`

Output: `[[1,4,7],[2,5,8],[3,6,9]]`

Example 2:

Input: `matrix = [[1,2,3],[4,5,6]]`

Output: `[[1,4],[2,5],[3,6]]`

Constraints:

- `m == matrix.length`

Code

Java ▾ Auto

```

1 class Solution {
2     public int[][] transpose(int[][] matrix) {
3         int rows=matrix.length;
4         int cols=matrix[0].length;
5         int [][] ans=new int[cols][rows];
6         for(int i=0;i<rows;i++){
7             for(int j=0;j<cols;j++){
8                 ans[j][i]=matrix[i][j];
9             }
10        }
11        return ans;
12    }
13 }
    
```

Saved

Ln 1, Col 1

☒ Testcase |
 >_ Test Result

Case 1

Case 2

+

matrix =

`[[1,2,3],[4,5,6],[7,8,9]]`

74. Search a 2D Matrix

Solved ✓

Medium

Topics

Companies

You are given an $m \times n$ integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` if `target` is in `matrix` or `false` otherwise.

You must write a solution in $O(\log(m * n))$ time complexity.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

Input: `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`,
`target = 3`

Code

Java ▾ 🔒 Auto

≡ |
 📖 |
 { } |
 ↶ |
 ↷

```

1  class Solution {
2      public boolean searchMatrix(int[][] matrix, int target) {
3          int rows = matrix.length;
4          int cols= matrix[0].length;
5          int left =0;
6          int right = rows * cols-1;
7          while (left<= right){
8              int mid = left + (right -left )/2;
9              int row = mid/ cols;
10             int col = mid %cols;
11             int current = matrix[row][col];
12             if (current == target){
13                 return true;
14             }
15             else if(current < target){
16                 left = mid+1;

```

Saved

Ln 1, Col 1

☒ Testcase |
 >_ Test Result

Case 1

Case 2

+

matrix =

`[[1,3,5,7],[10,11,16,20],[23,30,34,60]]`

target =

`3`

48. Rotate Image

Solved ✓

Medium

Topics

Companies

You are given an $n \times n$ 2D `matrix` representing an image, rotate the image by 90 degrees (clockwise).

You have to rotate the image **in-place**, which means you have to modify the input 2D matrix directly. **DO NOT** allocate another 2D matrix and do the rotation.

Example 1:

1	2	3		7	4	1
4	5	6		8	5	2
7	8	9		9	6	3

Input: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`

Output: `[[7,4,1],[8,5,2],[9,6,3]]`

Example 2:

5	1	9	11		15	13	2	5
2	4	8	10		14	3	4	1

Code

Java Auto

```

1 class Solution {
2     public void rotate(int[][] matrix) {
3         int n= matrix.length;
4         for ( int i=0; i<n; i++){
5             for ( int j=i+1; j<n; j++){
6                 int temp =matrix [i] [j];
7                 matrix [i] [j]= matrix [j] [i] ;
8                 matrix [j] [i] = temp;
9             }
10        }
11        for (int i = 0; i< n; i++) {
12            int left =0;int right = n-1;
13            while (left < right) {
14                int temp = matrix[i][left];
15                matrix[i] [left]= matrix [i] [right];
16                matrix[i] [right] = temp;

```

Saved

Ln 1, Col 1

Testcase Test Result

Case 1

Case 2

+

matrix =

`[[1,2,3],[4,5,6],[7,8,9]]`

20.2K

424

☆

?

99 Online

Source ?

706. Design HashMap

Easy

Topics

Companies

Design a HashMap without using any built-in hash table libraries.

Implement the `MyHashMap` class:

- `MyHashMap()` initializes the object with an empty map.
- `void put(int key, int value)` inserts a `(key, value)` pair into the HashMap. If the `key` already exists in the map, update the corresponding `value`.
- `int get(int key)` returns the `value` to which the specified `key` is mapped, or `-1` if this map contains no mapping for the `key`.
- `void remove(key)` removes the `key` and its corresponding `value` if the map contains the mapping for the `key`.

Example 1:

Input

```
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]
```

```
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
```

Output

```
[null, null, null, 1, -1, null, 1, null, -1]
```

Explanation

```
MyHashMap myHashMap = new MyHashMap();
myHashMap.put(1, 1); // The map is now [[1,1]]
myHashMap.put(2, 2); // The map is now [[1,1], [2,2]]
myHashMap.get(1);    // return 1, The map is now [[1,1], [2,2]]
```

Code

Java Auto

```

1 class MyHashMap {
2
3     int[] map;
4
5     public MyHashMap() {
6         map = new int[1000001];
7         for (int i = 0; i < map.length; i++) {
8             map[i] = -1;
9         }
10    }
11
12    public void put(int key, int value) {
13        map[key] = value;
14    }
15
16    public int get(int key) {

```

Saved

Ln 23, Col 2

☒ Testcase |
 >_ Test Result

Accepted

Runtime: 2 ms

☒ Case 1

Input

```
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]
```

```
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
```

Output

```
[null, null, null, 1, -1, null, 1, null, -1]
```